

MEGA-GUIDA CHINOOK — Esame TdP

Tutto quello che potrebbe uscire sul database Chinook, con soluzioni complete

Il prof ha annunciato che l'esame userà **Chinook** (lo stesso DB della simulazione sim1). Questa guida ti prepara a QUALSIASI grafo possa chiederti: ti dà lo schema, tutte le query di nodi/archi/pesi per Famiglia 1 e 2, i problemi più probabili con soluzione completa, e le varianti che il prof potrebbe inventare.

Strategia: Chinook ha tante tabelle collegate → il prof può costruire molti grafi diversi. Ma sono tutti combinazioni dei pattern che già conosci. Studia lo schema e le catene di relazioni: è la chiave per non bloccarti.

0. LO SCHEMA CHINOOK — le 11 tabelle

Chinook è un negozio di musica digitale. Ecco le tabelle e i campi che ti servono:

Tabella	Campi principali	Chiave primaria
artist	ArtistId, Name	ArtistId
album	AlbumId, Title, ArtistId	AlbumId
track	TrackId, Name, AlbumId, MediaTypeId, GenreId, Composer, Milliseconds, Bytes, UnitPrice	TrackId
genre	GenreId, Name	GenreId
mediatype	MediaTypeId, Name	MediaTypeId
playlist	PlaylistId, Name	PlaylistId
playlisttrack	PlaylistId, TrackId	(PlaylistId, TrackId)
invoice	InvoiceId, CustomerId, InvoiceDate, BillingCountry, Total	InvoiceId
invoiceline	InvoiceLineId, InvoiceId, TrackId, UnitPrice, Quantity	InvoiceLineId
customer	CustomerId, FirstName, LastName, Country, SupportRepId	CustomerId
employee	EmployeeId, FirstName, LastName, Title, ReportsTo, BirthDate	EmployeeId

I nomi delle tabelle possono variare (es. `playlist_track` invece di `playlisttrack`, `invoice_items` invece di `invoiceline`). **Controlla su DBeaver i nomi esatti all'inizio dell'esame!**

1. LE CATENE DI RELAZIONI — come muoversi nel DB

Questa è la cosa più importante da capire. Per collegare due tabelle lontane, devi seguire la "catena" di chiavi. Memorizza queste 5 catene:

Catena 1 — il catalogo musicale

```
artist → album → track → genre
ArtistId AlbumId GenreId
```

"Da un artista alle sue tracce e ai generi": `artist.ArtistId = album.ArtistId`, `album.AlbumId = track.AlbumId`, `track.GenreId = genre.GenreId`.

Catena 2 — le vendite

```
track → invoiceline → invoice → customer
TrackId InvoiceId CustomerId
```

"Da una traccia a chi l'ha comprata": `track.TrackId = invoiceline.TrackId , invoiceline.InvoiceId = invoice.InvoiceId , invoice.CustomerId = customer.CustomerId .`

Catena 3 — le playlist

```
track → playlisttrack → playlist
TrackId      PlaylistId
```

"Da una traccia alle playlist che la contengono": `track.TrackId = playlisttrack.TrackId , playlisttrack.PlaylistId = playlist.PlaylistId .`

Catena 4 — il supporto clienti

```
customer → employee (SupportRepId)
```

"Dal cliente al suo addetto vendite": `customer.SupportRepId = employee.EmployeeId .`

Catena 5 — la gerarchia aziendale

```
employee → employee (ReportsTo)
```

"Dall'impiegato al suo capo": `employee.ReportsTo = capo.EmployeeId .`

La catena COMPLETA (artista ↔ cliente)

Per collegare un artista a chi lo compra, unisci catena 1 + catena 2:

```
artist → album → track → invoiceline → invoice → customer
```

Questa è la catena più usata nei temi Chinook!

2. LE QUERY DROPDOWN — riempire i menu

```
-- generi musicali (il più comune)
SELECT DISTINCT g.Name AS nome FROM genre g ORDER BY g.Name;

-- paesi dei clienti
SELECT DISTINCT c.Country AS paese FROM customer c ORDER BY c.Country;

-- anni delle fatture (per un range temporale)
SELECT DISTINCT YEAR(i.InvoiceDate) AS anno FROM invoice i ORDER BY anno;

-- tipi di media (MP3, AAC...)
SELECT DISTINCT m.Name AS nome FROM mediatype m ORDER BY m.Name;

-- playlist
SELECT DISTINCT p.Name AS nome FROM playlist p ORDER BY p.Name;

-- range date fatture (per i DatePicker → first/last)
SELECT DISTINCT i.InvoiceDate FROM invoice i ORDER BY i.InvoiceDate;
```

3. LE QUERY NODI — tutti i casi possibili

I nodi possono essere artisti, tracce, clienti, album, generi, impiegati. Ecco le query.

```

-- ARTISTI di un genere (il classico)
SELECT DISTINCT art.*
FROM artist art, album al, track t, genre g
WHERE art.ArtistId = al.ArtistId AND t.AlbumId = al.AlbumId
      AND t.GenreId = g.GenreId AND g.Name = %s;

-- TRACCE di un genere
SELECT DISTINCT t.*
FROM track t, genre g
WHERE t.GenreId = g.GenreId AND g.Name = %s;

-- TRACCE di una playlist
SELECT DISTINCT t.*
FROM track t, playlisttrack pt, playlist p
WHERE t.TrackId = pt.TrackId AND pt.PlaylistId = p.PlaylistId AND p.Name = %s;

-- TRACCE di un media type
SELECT DISTINCT t.*
FROM track t, mediatype m
WHERE t.MediaTypeId = m.MediaTypeId AND m.Name = %s;

-- CLIENTI di un paese
SELECT * FROM customer c WHERE c.Country = %s;

-- CLIENTI che hanno comprato un genere
SELECT DISTINCT c.*
FROM customer c, invoice i, invoiceline il, track t, genre g
WHERE c.CustomerId = i.CustomerId AND i.InvoiceId = il.InvoiceId
      AND il.TrackId = t.TrackId AND t.GenreId = g.GenreId AND g.Name = %s;

-- ALBUM di un genere
SELECT DISTINCT al.*
FROM album al, track t, genre g
WHERE al.AlbumId = t.AlbumId AND t.GenreId = g.GenreId AND g.Name = %s;

-- GENERI (tutti)
SELECT * FROM genre;

-- IMPIEGATI (tutti)
SELECT * FROM employee;

```

Come scegli la query nodi? La traccia dice "i nodi sono X". Trovi la tabella di X e applichi il filtro richiesto seguendo la catena giusta. Sempre `DISTINCT` se la catena può creare ripetizioni.

4. LE QUERY ARCHI — FAMIGLIA 1 (metrica del singolo nodo)

Ricorda: Famiglia 1 = il peso/verso dipende da una **metrica del singolo nodo**. Query con UN id + metrica, poi `combinations` in Python.

```

-- POPOLARITÀ artista (tracce vendute) – per il classico verso "più popolare → meno popolare"
-- versione "per cliente" (serve per l'arco = cliente comune):
SELECT i.CustomerId AS cust, art.ArtistId AS art, COUNT(*) AS n
FROM invoice i, invoiceLine il, track t, album al, artist art, genre g
WHERE i.InvoiceId = il.InvoiceId AND il.TrackId = t.TrackId
      AND t.AlbumId = al.AlbumId AND al.ArtistId = art.ArtistId
      AND t.GenreId = g.GenreId AND g.Name = %s
GROUP BY i.CustomerId, art.ArtistId;

-- REVENUE per artista (somma incassi = UnitPrice * Quantity)
SELECT art.ArtistId AS id, SUM(il.UnitPrice * il.Quantity) AS revenue
FROM artist art, album al, track t, invoiceLine il
WHERE art.ArtistId = al.ArtistId AND al.AlbumId = t.AlbumId AND t.TrackId = il.TrackId
GROUP BY art.ArtistId;

-- NUMERO ALBUM per artista
SELECT art.ArtistId AS id, COUNT(*) AS numAlbum
FROM artist art, album al
WHERE art.ArtistId = al.ArtistId
GROUP BY art.ArtistId;

-- NUMERO TRACCE per artista
SELECT art.ArtistId AS id, COUNT(*) AS numTracce
FROM artist art, album al, track t
WHERE art.ArtistId = al.ArtistId AND al.AlbumId = t.AlbumId
GROUP BY art.ArtistId;

-- DURATA per traccia (già nel campo, nessun GROUP BY!)
SELECT t.TrackId AS id, t.Milliseconds AS durata FROM track t;

-- PREZZO per traccia (già nel campo)
SELECT t.TrackId AS id, t.UnitPrice AS prezzo FROM track t;

-- SPESA TOTALE per cliente
SELECT c.CustomerId AS id, SUM(i.Total) AS spesa
FROM customer c, invoice i
WHERE c.CustomerId = i.CustomerId
GROUP BY c.CustomerId;

```

5. LE QUERY ARCHI — FAMIGLIA 2 (relazione diretta)

Ricorda: Famiglia 2 = l'arco nasce da una **relazione diretta** nel DB. Self-join, produce coppie, poi `add_edge` in Python.

```

-- TRACCE nella stessa PLAYLIST (copresenza, self-join su playlisttrack)
SELECT pt1.TrackId AS t1, pt2.TrackId AS t2, COUNT(*) AS peso
FROM playlisttrack pt1, playlisttrack pt2
WHERE pt1.PlaylistId = pt2.PlaylistId
      AND pt1.TrackId < pt2.TrackId
GROUP BY t1, t2;
-- peso = numero di playlist in comune

-- TRACCE comprate insieme (stessa fattura)
SELECT il1.TrackId AS t1, il2.TrackId AS t2, COUNT(DISTINCT il1.InvoiceId) AS peso
FROM invoice il1, invoice il2
WHERE il1.InvoiceId = il2.InvoiceId
      AND il1.TrackId < il2.TrackId
GROUP BY t1, t2;
-- peso = numero di fatture in cui compaiono insieme

-- CLIENTI che hanno comprato lo stesso ARTISTA (self-join via catena completa)
SELECT i1.CustomerId AS c1, i2.CustomerId AS c2, COUNT(DISTINCT al1.ArtistId) AS peso
FROM invoice i1, invoice i2, invoice il1, invoice il2, track t1, track t2, album al1,
      album al2
WHERE i1.InvoiceId = i2.InvoiceId AND il1.InvoiceId = i1.InvoiceId AND il2.InvoiceId = i2.InvoiceId
      AND t1.TrackId = t2.TrackId AND al1.AlbumId = al2.AlbumId
      AND i1.CustomerId < i2.CustomerId
      AND al1.ArtistId = al2.ArtistId
      -- stesso artista (il ponte)
      -- coppia distinta di clienti
GROUP BY c1, c2;
-- peso = numero di artisti condivisi

-- CLIENTI che hanno comprato la stessa TRACCIA
SELECT i1.CustomerId AS c1, i2.CustomerId AS c2, COUNT(DISTINCT il1.TrackId) AS peso
FROM invoice i1, invoice i2, invoice il1, invoice il2
WHERE i1.InvoiceId = i2.InvoiceId AND il1.InvoiceId = i1.InvoiceId AND il2.InvoiceId = i2.InvoiceId
      AND il1.TrackId = il2.TrackId
      -- stessa traccia
      AND i1.CustomerId < i2.CustomerId
GROUP BY c1, c2;

-- CLIENTI dello stesso SUPPORT REP
SELECT c1.CustomerId AS c1, c2.CustomerId AS c2
FROM customer c1, customer c2
WHERE c1.SupportRepId = c2.SupportRepId
      AND c1.CustomerId < c2.CustomerId;

-- CLIENTI dello stesso PAESE
SELECT c1.CustomerId AS c1, c2.CustomerId AS c2
FROM customer c1, customer c2
WHERE c1.Country = c2.Country
      AND c1.CustomerId < c2.CustomerId;

-- ARTISTI comprati dallo stesso CLIENTE (self-join via catena)
SELECT al1.ArtistId AS a1, al2.ArtistId AS a2, COUNT(DISTINCT i1.CustomerId) AS peso
FROM invoice i1, invoice i2, invoice il1, invoice il2, track t1, track t2, album al1,
      album al2
WHERE i1.InvoiceId = i2.InvoiceId AND il1.InvoiceId = i1.InvoiceId AND il2.InvoiceId = i2.InvoiceId
      AND t1.TrackId = t2.TrackId AND al1.AlbumId = al2.AlbumId
      AND al1.ArtistId < al2.ArtistId
GROUP BY a1, a2;
-- peso = numero di clienti che hanno comprato entrambi

-- GENERI comprati dallo stesso CLIENTE
SELECT g1.GenreId AS g1, g2.GenreId AS g2, COUNT(DISTINCT i1.CustomerId) AS peso
FROM invoice i1, invoice i2, invoice il1, invoice il2, track t1, track t2, genre g1,
      genre g2
WHERE i1.InvoiceId = i2.InvoiceId AND il1.InvoiceId = i1.InvoiceId AND il2.InvoiceId = i2.InvoiceId
      AND t1.TrackId = t2.TrackId AND g1.GenreId = g2.GenreId
      AND g1.GenreId < g2.GenreId
GROUP BY g1, g2;

-- IMPIEGATI con lo stesso CAPO (ReportsTo)
SELECT e1.EmployeeId AS e1, e2.EmployeeId AS e2
FROM employee e1, employee e2
WHERE e1.ReportsTo = e2.ReportsTo
      AND e1.EmployeeId < e2.EmployeeId;

```

6. I PESI POSSIBILI — come riconoscerli

La traccia dice...	Peso	Famiglia	Dove calcolarlo
"numero di clienti in comune"	<code>COUNT(DISTINCT customer)</code>	2	SQL
"numero di playlist in comune"	<code>COUNT(*)</code> self-join playlist	2	SQL
"numero di tracce in comune"	<code>COUNT(DISTINCT track)</code>	2	SQL
"incasso totale / fatturato"	<code>SUM(UnitPrice * Quantity)</code>	1	SQL o Python
"somma delle popolarità"	tracce vendute totali	1	Python
"durata totale"	<code>SUM(Milliseconds)</code>	1	Python
"somma vendite dei due"	metrica per nodo	1	Python

Regola chiave: peso = COUNT/SUM della **coppia** (clienti comuni, playlist comuni) → Famiglia 2, in SQL. Peso = metrica **totale del singolo nodo** (popolarità, fatturato dell'artista) → Famiglia 1, in Python.

7. PROBLEMI COMPLETI CON SOLUZIONE

Qui hai i problemi più probabili, già risolti per intero. Studiali: l'esame sarà una variazione di questi.

PROBLEMA A — Il classico (sim1): artisti per genere, arco = cliente comune, verso = popolarità

Traccia tipo: "I nodi sono gli artisti di un genere. Due artisti sono collegati se lo stesso cliente ha comprato tracce di entrambi. Il verso va dall'artista più popolare (più tracce vendute in totale) al meno popolare. Il peso è la somma delle popolarità. In caso di parità, due archi."

Questo è un **IBRIDO**: arco da copresenza (Famiglia 2 via cliente), verso/peso da popolarità (Famiglia 1).

DAO:

```
@staticmethod
def getArtistiByGenere(genere):
    conn = DBConnect.get_connection()
    results = []
    cursor = conn.cursor(dictionary=True)
    query = """SELECT DISTINCT art.*
                FROM artist art, album al, track t, genre g
                WHERE art.ArtistId = al.ArtistId AND t.AlbumId = al.AlbumId
                AND t.GenreId = g.GenreId AND g.Name = %s"""
    cursor.execute(query, (genere,))
    for row in cursor:
        results.append(Artist(**row))
    cursor.close(); conn.close()
    return results

@staticmethod
def getClienteArtista(genere):
    conn = DBConnect.get_connection()
    results = []
    cursor = conn.cursor(dictionary=True)
    query = """SELECT i.CustomerId AS cust, art.ArtistId AS art, COUNT(*) AS n
                FROM invoice i, invoiceline il, track t, album al, artist art, genre g
                WHERE i.InvoiceId = il.InvoiceId AND il.TrackId = t.TrackId
                AND t.AlbumId = al.AlbumId AND al.ArtistId = art.ArtistId
                AND t.GenreId = g.GenreId AND g.Name = %s
                GROUP BY i.CustomerId, art.ArtistId"""
    cursor.execute(query, (genere,))
    for row in cursor:
        results.append((row["cust"], row["art"], row["n"]))
    cursor.close(); conn.close()
    return results
```

Model:

```

from itertools import combinations
import networkx as nx
from database.DAO import DAO

class Model:
    def __init__(self):
        self._graph = nx.DiGraph()
        self._idMap = {}
        self._pop = {}

    def buildGraph(self, genere):
        self._graph.clear(); self._idMap.clear(); self._pop.clear()
        # NODI
        artisti = DAO.getArtistiByGenere(genere)
        for a in artisti:
            self._idMap[a.ArtistId] = a
        self._graph.add_nodes_from(artisti)
        # dati cliente-artista
        righe = DAO.getClientiArtista(genere)
        # 1) mappa {cliente: {artista: n}}
        customerMap = {}
        for cust, art, n in righe:
            if cust not in customerMap:
                customerMap[cust] = {}
            customerMap[cust][art] = n
        # 2) popolarità = somma tracce su tutti i clienti
        for cust, artNm in customerMap.items():
            for art, n in artNm.items():
                self._pop[art] = self._pop.get(art, 0) + n
        # 3) per ogni cliente, ogni coppia di artisti → arco orientato per popolarità
        for cust, artNm in customerMap.items():
            for a, b in combinations(artNm.keys(), 2):
                pa, pb = self._pop[a], self._pop[b]
                peso = pa + pb
                if pa > pb:
                    self._graph.add_edge(self._idMap[a], self._idMap[b], weight=peso)
                elif pa < pb:
                    self._graph.add_edge(self._idMap[b], self._idMap[a], weight=peso)
                else:
                    self._graph.add_edge(self._idMap[a], self._idMap[b], weight=peso)
                    self._graph.add_edge(self._idMap[b], self._idMap[a], weight=peso)

    def graphDetails(self):
        return len(self._graph.nodes), len(self._graph.edges)

```

Perché non si contano doppi gli archi? La popolarità è globale (somma su tutti i clienti). Se due artisti sono comprati insieme da più clienti, `add_edge` sovrascrive lo stesso peso. Nessun doppio conteggio.

PROBLEMA B — Tracce stessa playlist, peso = playlist comuni (FAMIGLIA 2 PURA)

Traccia tipo: "I nodi sono le tracce di un genere. Due tracce sono collegate se compaiono in almeno una playlist insieme. Il peso è il numero di playlist in comune." Grafo non orientato.

DAO:

```

@staticmethod
def getTracceByGenere(genere):
    # ... SELECT DISTINCT t.* FROM track t, genere g WHERE t.GenreId=g.GenreId AND g.Name=%s
    # ritorna Track(**row)

@staticmethod
def getArchiPlaylist():
    conn = DBConnect.get_connection()
    results = []
    cursor = conn.cursor(dictionary=True)
    query = """SELECT pt1.TrackId AS t1, pt2.TrackId AS t2, COUNT(*) AS peso
                FROM playlisttrack pt1, playlisttrack pt2
                WHERE pt1.PlaylistId = pt2.PlaylistId AND pt1.TrackId < pt2.TrackId
                GROUP BY t1, t2"""
    cursor.execute(query)
    for row in cursor:
        results.append((row["t1"], row["t2"], row["peso"]))
    cursor.close(); conn.close()
    return results

```

Model (Famiglia 2 = solo add_edge):

```
def buildGraph(self, genere):
    self._graph.clear(); self._idMap.clear()
    nodi = DAO.getTracceByGenere(genere)
    for t in nodi:
        self._idMap[t.TrackId] = t
    self._graph.add_nodes_from(nodi)
    archi = DAO.getArchiPlaylist()
    for t1, t2, peso in archi:
        if t1 in self._idMap and t2 in self._idMap: # solo se entrambi nel grafo!
            self._graph.add_edge(self._idMap[t1], self._idMap[t2], weight=peso)
```

Nota il check `if t1 in self._idMap and t2 in self._idMap`: la query archi prende TUTTE le coppie di playlist, ma tu vuoi solo quelle tra le tracce del genere selezionato.

PROBLEMA C — Clienti stesso artista, peso = artisti condivisi (FAMIGLIA 2)

Traccia tipo: "I nodi sono i clienti di un paese. Due clienti sono collegati se hanno comprato tracce dello stesso artista. Il peso è il numero di artisti in comune." Grafo non orientato.

DAO archi:

```
@staticmethod
def getArchiClientiArtista():
    conn = DBConnect.get_connection()
    results = []
    cursor = conn.cursor(dictionary=True)
    query = """SELECT i1.CustomerId AS c1, i2.CustomerId AS c2, COUNT(DISTINCT al1.ArtistId) AS peso
    FROM invoice i1, invoiceline il1, track t1, album al1,
    invoice i2, invoiceline il2, track t2, album al2
    WHERE i1.InvoiceId = il1.InvoiceId AND il1.TrackId = t1.TrackId AND t1.AlbumId = al1.AlbumId
    AND i2.InvoiceId = il2.InvoiceId AND il2.TrackId = t2.TrackId AND t2.AlbumId = al2.AlbumId
    AND al1.ArtistId = al2.ArtistId
    AND i1.CustomerId < i2.CustomerId
    GROUP BY c1, c2"""
    cursor.execute(query)
    for row in cursor:
        results.append((row["c1"], row["c2"], row["peso"]))
    cursor.close(); conn.close()
    return results
```

Model: identico al Problema B — nodi (clienti del paese) + add_edge sulle coppie (con check `if in idMap`).

PROBLEMA D — Variante ibrida (come UFO): clienti stesso paese, peso = spesa totale

Traccia tipo (variante inventata): "I nodi sono i clienti che hanno comprato un genere. Due clienti sono collegati se sono dello stesso paese. Il peso è la somma della spesa totale dei due clienti (su tutti gli acquisti)."

IBRIDO: arco = stesso paese (Famiglia 2), peso = spesa totale del singolo cliente (Famiglia 1).

DAO:

```
@staticmethod
def getSpesaCliente():
    # SELECT c.CustomerId AS id, SUM(i.Total) AS spesa
    # FROM customer c, invoice i WHERE c.CustomerId=i.CustomerId GROUP BY c.CustomerId
    # ritorna tuple (id, spesa)

@staticmethod
def getArchiStessoPaese():
    # SELECT c1.CustomerId AS c1, c2.CustomerId AS c2
    # FROM customer c1, customer c2 WHERE c1.Country=c2.Country AND c1.CustomerId<c2.CustomerId
    # ritorna tuple (c1, c2)
```

Model (due mappe, come UFO):


```

def buildGraph(self, genere):
    self._graph.clear(); self._idMap.clear(); self._spesa.clear()
    # nodi
    nodi = DAO.getClientiByGenere(genere)
    for c in nodi:
        self._idMap[c.CustomerId] = c
    self._graph.add_nodes_from(nodi)
    # metrica (spesa) per nodo
    for id, spesa in DAO.getSpesaCliente():
        self._spesa[id] = spesa
    # archi: coppie stesso paese, peso = spesa1 + spesa2
    for c1, c2 in DAO.getArchiStessoPaese():
        if c1 in self._idMap and c2 in self._idMap:
            peso = self._spesa.get(c1, 0) + self._spesa.get(c2, 0)
            self._graph.add_edge(self._idMap[c1], self._idMap[c2], weight=peso)

```

8. PUNTO C/D — le stampe possibili su Chinook

Dopo "Crea grafo", il prof chiede dettagli. Ecco i metodi pronti.

```

# numero nodi e archi
def graphDetails(self):
    return len(self._graph.nodes), len(self._graph.edges)

# top K archi per peso
def getTopKArchi(self, k=5):
    return sorted(self._graph.edges(data=True),
                  key=lambda x: x[2]["weight"], reverse=True)[:k]

# componenti connesse (Graph non orientato)
def getComponenti(self):
    return nx.number_connected_components(self._graph)

# componenti DEBOLI (DiGraph! es. il problema A è orientato)
def getComponentiDeboli(self):
    return nx.number_weakly_connected_components(self._graph)

# componente più grande, nodi ordinati per grado
def getMaggioreOrdinata(self):
    largest = max(nx.connected_components(self._graph), key=len)
    ordinati = sorted(largest, key=lambda n: self._graph.degree(n), reverse=True)
    return [(n, self._graph.degree(n)) for n in ordinati]

# ARTISTA PIÙ POPOLARE (DiGraph): max influenza = peso uscenti - entranti
def getBestNode(self):
    best, bestScore = None, None
    for v in self._graph.nodes():
        out = sum(d["weight"] for _, _, d in self._graph.out_edges(v, data=True))
        inn = sum(d["weight"] for _, _, d in self._graph.in_edges(v, data=True))
        score = out - inn
        if bestScore is None or score > bestScore:
            bestScore, best = score, v
    return best, bestScore

# vicini di un nodo, ordinati per peso
def getVicini(self, source):
    out = [(v, self._graph[source][v]["weight"]) for v in self._graph.neighbors(source)]
    out.sort(key=lambda x: x[1], reverse=True)
    return out

# top K nodi per grado
def getTopKNodi(self, k=5):
    return sorted([(n, self._graph.degree(n)) for n in self._graph.nodes()],
                  key=lambda x: x[1], reverse=True)[:k]

```

Se il grafo è **orientato** (Problema A), per le componenti usa `weakly_connected_components`, NON `connected_components` !

9. PUNTO 2 — RICORSIONE su Chinook (bonus, se ci arrivi)

Anche se punti alla sufficienza, ecco gli schemi di ricorsione che il prof potrebbe chiedere. Tutti la stessa struttura: aggiorna best → estendi → backtrack.

Cammino di peso crescente (su DiGraph orientato — Problema A)

```
import copy
def trovaCammino(self, sourceId):
    self._best = []
    source = self._idMap[int(sourceId)]
    self._ricorsione([source], 0)
    return self._best

def _ricorsione(self, parziale, lastWeight):
    if len(parziale) > len(self._best):
        self._best = copy.deepcopy(parziale)
    current = parziale[-1]
    for _, succ, data in self._graph.out_edges(current, data=True):
        w = data["weight"]
        if w >= lastWeight and succ not in parziale: # peso crescente + non già visitato
            parziale.append(succ)
            self._ricorsione(parziale, w)
            parziale.pop()
```

Cammino più lungo (grafo non orientato)

```
def _ricorsione(self, parziale):
    if len(parziale) > len(self._best):
        self._best = copy.deepcopy(parziale)
    current = parziale[-1]
    for vicino in self._graph.neighbors(current):
        if vicino not in parziale:
            parziale.append(vicino)
            self._ricorsione(parziale)
            parziale.pop()
```

Insieme di K nodi (uno per componente, ottimizzando qualcosa)

```
def trovaSet(self, k):
    self._opt = []; self._optScore = float('inf')
    comps = list(nx.connected_components(self._graph))
    if k > len(comps):
        return None, 0
    self._ric(comps, k, [], 0)
    return self._opt, self._optScore

def _ric(self, comps, k, parziale, idx):
    if len(parziale) == k:
        score = self._calcola(parziale) # es. somma pesi, range...
        if score < self._optScore:
            self._optScore = score
            self._opt = copy.deepcopy(parziale)
        return
    if idx >= len(comps): return
    if (len(comps) - idx) < (k - len(parziale)): return # pruning
    # salto questo componente
    self._ric(comps, k, parziale, idx + 1)
    # scelgo un nodo da questo componente
    for nodo in comps[idx]:
        parziale.append(nodo)
        self._ric(comps, k, parziale, idx + 1)
        parziale.pop()
```

10. CHECKLIST E CONSIGLI PER CHINOOK

All'inizio dell'esame: - [] Controlla i nomi ESATTI delle tabelle su DBeaver ([playlisttrack](#) o [playlist_track](#) ?
[invoice](#) o [invoice_items](#) ?) - [] Controlla i nomi delle colonne (maiuscole/minuscole) - [] Identifica subito: nodi = cosa?
arco = Famiglia 1 o 2? peso = del nodo o della coppia?

Le catene da ricordare: - Artista → cliente: `artist → album → track → invoice → customer` - Traccia → playlist: `track → playlist` - Cliente → impiegato: `customer.SupportRepId = employee.EmployeeId`

I 3 problemi-tipo: - **A** (ibrido): artisti per genere, arco = cliente comune, verso = popolarità → DiGraph + combinations - **B** (Fam. 2 pura): tracce stessa playlist, peso = playlist comuni → add_edge - **C** (Fam. 2): clienti stesso artista, peso = artisti condivisi → add_edge - **D** (ibrido): stesso paese + spesa totale → due mappe (come UFO)

Gli errori da non fare: - [] Dimenticare il check `if id in idMap` quando la query archi prende coppie globali - [] Usare `connected_components` su un DiGraph (Problema A) → usa `weakly` - [] Sbagliare la catena (saltare una tabella nel JOIN) - [] `combinations` quando è Famiglia 2 (non serve, le coppie sono già in SQL) - [] Non mettere `DISTINCT` sui nodi (la catena crea ripetizioni)

RIEPILOGO — Chinook in 5 punti

1. **Schema:** 11 tabelle, 5 catene. La catena chiave è artista↔cliente (album→track→invoice→customer).
2. **Nodi:** artisti/tracce/clienti filtrati per genere/paese/playlist. Sempre `DISTINCT`.
3. **Famiglia 1** (popolarità, revenue, durata): metrica per nodo → combinations in Python.
4. **Famiglia 2** (cliente comune, playlist comune, stesso artista): self-join → add_edge.
5. **Il classico** è il Problema A (ibrido orientato). Ma preparati anche a B, C, D.

Controlla SEMPRE i nomi delle tabelle su DBeaver all'inizio, e testa ogni query prima di portarla nel DAO.

In bocca al lupo Armi — con Chinook hai tutte le carte in regola!